



HowTo: RPC

☰ Tags	OP
# Cap	-99
⚙ Status	Done
🕒 Ultima modifica	@February 16, 2024 3:14 AM

Info di base

Documentazione per l'Utilizzo e la Creazione di Applicazioni con RPC

Introduzione

Cos'è RPC?

Utilizzo di RPC

Creazione di Applicazioni con RPC

Esempio di Utilizzo di RPC

Definizione dell'Interfaccia

Generazione del Codice Stub

Esecuzione del makefile

Implementazione del Server (in C)

Implementazione del Client (in C)

Conclusioni

Info di base

Documentazione per l'Utilizzo e la Creazione di Applicazioni con RPC

Introduzione

RPC (Remote Procedure Call) è un protocollo che consente a un programma di richiamare procedure o funzioni su un sistema remoto come se fossero locali. Questa documentazione fornisce una panoramica sull'utilizzo di RPC e spiega come creare applicazioni utilizzando questo protocollo.

Cos'è RPC?

RPC è un meccanismo di comunicazione che consente a un programma di eseguire codice su un altro computer o processo, come se fosse eseguito localmente. Utilizza un'interfaccia simile a una chiamata di procedura locale per invocare funzioni su un sistema remoto.

Utilizzo di RPC

Per utilizzare RPC, è necessario seguire i seguenti passaggi:

1. **Definizione delle Interfacce:** Prima di poter utilizzare RPC, è necessario definire le interfacce delle procedure che si desidera chiamare remotamente. Queste interfacce descrivono i parametri delle procedure e il loro tipo di ritorno.
2. **Generazione del Codice Stub:** Dopo aver definito le interfacce, è necessario generare il codice stub per le interfacce client e server. Il codice stub funge da intermediario tra il client e il server, gestendo la comunicazione RPC.
3. **Implementazione del Server:** Il server implementa le procedure definite nell'interfaccia e ascolta le richieste RPC dai client.
4. **Implementazione del Client:** Il client utilizza il codice stub generato per chiamare le procedure sul server remoto.

Creazione di Applicazioni con RPC

Per creare un'applicazione utilizzando RPC, è possibile seguire questi passaggi di base:

1. **Definizione delle Interfacce:** Definire le interfacce delle procedure che saranno chiamate tramite RPC.
2. **Generazione del Codice Stub:** Utilizzare strumenti come `rpcgen` per generare il codice stub per il client e il server.

3. **Implementazione del Server:** Implementare le procedure definite nell'interfaccia sul server utilizzando il linguaggio di programmazione e le librerie RPC appropriate.
4. **Implementazione del Client:** Implementare il client utilizzando il codice stub generato e chiamare le procedure remote come se fossero locali.

Esempio di Utilizzo di RPC

Di seguito è riportato un esempio di utilizzo di RPC per la somma di due numeri:

Definizione dell'Interfaccia

```
/* File: add.x */
struct due_int {
    int numero1;
    int numero2;
};

program ADDPROG {
    version ADDVERSION {
        int ADD(struct due_int) = 1;
        int MULTIPLY(struct due_int) = 2;
    } = 1;
} = 0x20000001;
```

Generazione del Codice Stub

```
rpcgen -a -C add.x
```

Esecuzione del makefile

```
make -f Makefile.add
```

Implementazione del Server (in C)

```
/*
 * This is sample code generated by rpcgen.
```

```

* These are only templates and you can use them
* as a guideline for developing your own functions.
*/

#include "add.h"

int *
add_1_svc(struct due_int *argp, struct svc_req *rqstp)
{
    static int    result;

    result = argp->numero1 + argp->numero2;

    return &result;
}

int *
multiply_1_svc(struct due_int *argp, struct svc_req *rqstp)
{
    static int    result;

    result = argp->numero1 * argp->numero2;

    return &result;
}

```

Implementazione del Client (in C)

```

/*
* This is sample code generated by rpcgen.
* These are only templates and you can use them
* as a guideline for developing your own functions.
*/

#include "add.h"

void

```

```

addprog_1(char *host)
{
    CLIENT *clnt;
    int *result_1;
    struct due_int add_1_arg;
    int *result_2;
    struct due_int multiply_1_arg;

#ifdef DEBUG
    clnt = clnt_create (host, ADDPROG, ADDVERSION, "udp");
    if (clnt == NULL) {
        clnt_pcreateerror (host);
        exit (1);
    }
#endif /* DEBUG */

    add_1_arg.numero1 = 10;
    add_1_arg.numero2 = 323;

    result_1 = add_1(&add_1_arg, clnt);
    if (result_1 == (int *) NULL) {
        clnt_perror (clnt, "call failed");
    }

    printf("%d\n", *result_1);

    multiply_1_arg.numero1 = 3;
    multiply_1_arg.numero2 = 4;

    result_2 = multiply_1(&multiply_1_arg, clnt);
    if (result_2 == (int *) NULL) {
        clnt_perror (clnt, "call failed");
    }

    printf("%d\n", *result_2);
#ifdef DEBUG
    clnt_destroy (clnt);
#endif /* DEBUG */
}

```

```

}

int
main (int argc, char *argv[])
{
    char *host;

    if (argc < 2) {
        printf ("usage: %s server_host\n", argv[0]);
        exit (1);
    }
    host = argv[1];
    addprog_1 (host);
    exit (0);
}

```

Conclusioni

RPC è un potente strumento per la creazione di applicazioni distribuite che richiedono la comunicazione tra processi remoti. Seguendo i passaggi descritti in questa documentazione, è possibile utilizzare e creare applicazioni con RPC in modo efficace.

Per ulteriori informazioni e dettagli sull'implementazione di RPC con un linguaggio specifico, consultare la documentazione e le risorse appropriate per quel linguaggio.